

TheLevelTwo

射击游戏—作业报告

程序设计实习课程作业

项目名称： TheLevelTwo 竖版射击游戏
开发工具： Qt 6 / C++17 + CMake
版本控制： Git + GitHub
小组成员： 陈莹蔚, 张怡馨, 李嘉明

2026 年 7 月 6 日

目录

1	程序功能介绍	2
1.1	核心玩法	2
1.2	辅助功能	2
2	项目各模块与类设计细节	3
2.1	项目结构总览	3
2.2	各模块与类设计	3
2.2.1	游戏引擎核心—MainWindow	3
2.2.2	游戏实体基类—GameObject	4
2.2.3	玩家—Player	4
2.2.4	敌人—Enemy & BossEnemy	4
2.2.5	子弹系统—Bullet & EnemyBullet	5
2.2.6	武器系统—Weapon	5
2.2.7	掉落物—PowerUp	5
2.2.8	宝箱与升级弹窗—Chest	5
2.2.9	视觉反馈—ParticleEffect & FloatingText	5
2.2.10	音频管理—AudioManager	6
2.2.11	存档管理—SaveManager	6
3	小组成员分工情况	6
4	项目总结与反思	6
4.1	项目成果	6
4.2	待改进之处	7
4.3	团队协作反思	7
4.4	学习收获	7

1 程序功能介绍

TheLevelTwo 是一款基于 Qt 6 / C++17 开发的竖版卷轴射击小游戏。玩家操控屏幕底部的飞船，向上方射击不断下降的敌人，通过层层关卡并挑战 BOSS，最终获取高分。

1.1 核心玩法

- **移动与射击：**使用 A/D 键或方向键控制飞船左右移动，空格键射击，操作简洁直观。
- **武器系统：**提供三种风格迥异的武器，按数字键 1/2/3 自由切换：
 - **枪 (Gun)：**初始武器，水平多发射击，均衡稳定。
 - **金箍棒 (GoldenCudgel)：**发射后悬停在命中位置，对范围内敌人造成持续伤害，适合清理密集敌群。
 - **绣花针 (EmbroideryNeedle)：**单根发射，击中敌人后扇状分裂为多根小针，分裂后的子针可继续分裂，适合穿透多名敌人。
- **宝箱升级系统：**从第 2 波开始有概率掉落宝箱，击毁宝箱后弹出升级选择窗口，每次随机生成 3 个升级选项（伤害、子弹数量、攻速、穿透、暴击率、武器专属强化等），品质分为普通/稀有/史诗三档。
- **道具拾取：**击败敌人有概率掉落回血（绿色十字）和武器升级（黄色方块）道具。
- **Boss 战：**每 3 波小怪后进入 Boss 关卡，Boss 拥有扇形弹幕攻击，难度随关卡递进（弹幕数量增多、出现瞄准弹等）。
- **关卡递进：**击败 Boss 后进入下一关，敌人属性全面提升，Boss 弹幕花样增加。

1.2 辅助功能

- **完整的菜单系统：**主菜单（开始游戏/设置/退出）、暂停菜单（继续/重新开始/返回菜单）、游戏结束菜单（显示分数，重新开始/返回菜单）。
- **设置系统：**可调节音效音量、背景音乐音量、全屏/窗口模式切换，设置自动持久化保存。
- **最高分存档：**基于 JSON 的本地存档系统，自动记录历史最高分。
- **视听反馈：**
 - 粒子特效（击杀敌人爆炸粒子、Boss 光环、道具拾取环状粒子等）
 - 浮动文字（伤害数值、得分、Boss 击败提示、武器升级提示）
 - 背景音乐 + 5 种音效（射击、受击、拾取、Boss 警报、开箱）
- **键盘 + 鼠标双操作：**支持键盘快捷键和鼠标点击按钮。

2 项目各模块与类设计细节

2.1 项目结构总览

```
TheLevelTwo/
|-- CMakeLists.txt           # CMake 构建配置 (Qt6 + C++17)
|-- README.md               # 项目说明
|-- Assets/audio/           # 音频资源 (6 个 .wav 文件)
|-- Codes/
|   |-- main.cpp            # 程序入口
|   |-- mainwindow.h / .cpp / .ui  # 主窗口 & 游戏引擎核心
|   |-- GameObject.h        # 游戏实体基类
|   |-- Player.h            # 玩家飞船
|   |-- Enemy.h             # 普通敌人
|   |-- Bullet.h            # 玩家子弹
|   |-- EnemyBullet.h       # 敌方子弹
|   |-- BossEnemy.h         # Boss 敌人
|   |-- Weapon.h            # 武器系统 (3 种武器)
|   |-- PowerUp.h           # 掉落道具
|   |-- Chest.h             # 宝箱 & 升级弹窗
|   |-- ParticleEffect.h    # 粒子特效
|   |-- FloatingText.h      # 浮动文字
|   |-- AudioManager.h      # 音频管理
|   |-- SaveManager.h       # 存档管理
|-- 文档合集/
|   |-- 射击小游戏设计文档.pdf
|   |-- 射击小游戏进展报告.pptx
|   |-- 游戏内容细节部分.md
```

2.2 各模块与类设计

2.2.1 游戏引擎核心—MainWindow

MainWindow 继承自 QMainWindow，是整个游戏的驱动核心，负责：

- **游戏状态机**: 定义 GameState 枚举 (Menu / Settings / Playing / Paused / GameOver)，所有逻辑和渲染按当前状态分派。
- **游戏循环**: 通过 QTimer 以约 16ms (60FPS) 周期驱动 gameLoop() 槽函数，使用 QElapsedTimer 计算 deltaTime，保证帧率无关的平滑运动。

- **玩家输入:** 通过 `keyPressEvent/keyReleaseEvent` 维护按键状态布尔量, 在游戏循环中根据状态平滑移动玩家。
- **敌人生成:** `spawnWave()` 按关卡难度生成敌人波次, 3 波后触发 `startBossBattle()`。
- **碰撞检测:** `checkCollisions()` 负责七类碰撞判定——子弹 vs 宝箱/敌人/Boss、玩家 vs 敌人/Boss、敌方子弹 vs 玩家、道具 vs 玩家。
- **渲染系统:** `paintEvent()` 按层绘制背景 → 实体 → 粒子 → 浮动文字 → HUD → 菜单遮罩, 所有图形均使用 `QPainter` 过程化绘制。
- **暂停控制:** `stopGameTimer()/startGameTimer()` 控制游戏循环启停, 宝箱升级弹窗时暂停游戏。

2.2.2 游戏实体基类—GameObject

所有游戏实体的抽象基类, 提供通用属性与方法:

- **属性:** `m_position` (`QPointF` 坐标)、`m_speed` (速度)、`m_hitbox` (碰撞箱 `QRectF`)。
- **方法:** `moveLeft/Right/Up/Down(deltaTime)` 四个方向移动 (含默认实现), 虚析构函数保证多态安全。

2.2.3 玩家—Player

- 碰撞箱 112×72 , 速度 300 px/s , 受屏幕边界约束。
- 持有 `std::unique_ptr<Weapon>` 实现武器多态, 通过 `setWeaponType()` 切换武器类型。
- 提供缩小的碰撞判定区域 (`getCollisionBox()`), 实现更公平的受击体验。

2.2.4 敌人—Enemy & BossEnemy

普通敌人 (Enemy):

- 碰撞箱 118×56 , 生命值公式 $1 + \text{wave} / 3$, 得分 $100 * \text{wave}$ 。
- 30% 概率掉落回血道具。
- 绘制为装饰性木纹方块, 显示波次编号。

Boss 敌人 (BossEnemy):

- 体积 180×105 , 生命值 $10 + \text{wave} * 2$, 得分 $1000 + \text{wave} * 200$ 。
- 入场动画从顶部降落后水平震荡移动, 头顶显示血条。
- 扇形弹幕攻击 (发射间隔 $2.4 - 0.18 * (\text{level}-1)$ 秒), 随关卡等级增加弹幕数量和花样。

2.2.5 子弹系统—Bullet & EnemyBullet

玩家子弹（Bullet）：8×15 像素，向上飞行 500 px/s，支持伤害、穿透次数、暴击属性。

敌方子弹（EnemyBullet）：12×12 像素圆形，沿方向向量飞行，超出屏幕自动清除。

2.2.6 武器系统—Weapon

以抽象基类 Weapon 为核心，通过子类多态实现三种武器：

表 1: 三种武器对比

武器	攻击间隔	基础伤害	特色机制
Gun	0.25s	1	水平多列齐射，均衡输出
GoldenCudgel	2.0s	3	悬停持续伤害,无限穿透,在场 3s
EmbroideryNeedle	1.5s	3	击中后 60° 扇形分裂 4 根子针

武器系统采用虚拟升级接口（upgradeDamage()、upgradeBulletCount() 等），支持通用升级与武器专属升级。clone() 纯虚函数实现多态深拷贝。

2.2.7 掉落物—PowerUp

- Heal（绿色十字）：回复 1 点生命值，生命上限 5。
- WeaponUpgrade（黄色方块）：提升武器等级，最高 3 级。
- 下降速度 120 px/s，尺寸 28×28。

2.2.8 宝箱与升级弹窗—Chest

- 宝箱：52×48 像素，80 px/s 下降，生命值 2 + wave / 2，第 2 波起约 35% 概率生成。
- UpgradeDialog（继承 QDialog）：300×400 模态窗口，深色主题，在 15+ 种升级选项中按加权随机选择 3 个供玩家选择（品质权重 6:3:1，高波次史诗加成）。

2.2.9 视觉反馈—ParticleEffect & FloatingText

- ParticleEffect：提供 explosion()（爆炸散开）和 ring()（环形扩散）两种工厂方法，粒子随生命值衰减淡出。

- **FloatingText:** 在指定位置显示浮动文字（得分”+100”、受伤”-1 HP”、Boss 击败”BOSS CLEAR” 等），向上飘动 1 秒后消失。

2.2.10 音频管理—AudioManager

- 使用 QMediaPlayer + QAudioOutput 播放背景音乐（循环）。
- 使用 QSoundEffect 播放 5 种音效（射击、受击、拾取、Boss 警报、开箱）。
- 支持音量独立调节（0-100），文件缺失时优雅降级。

2.2.11 存档管理—SaveManager

- 纯静态类，使用 QJsonDocument 读写 save.json。
- 持久化字段:highScore、sfxVolume（默认 70）、musicVolume（默认 60）、fullscreen（默认 false）。
- 存储路径使用 QStandardPaths::AppDataLocation，保证跨平台兼容。

3 小组成员分工情况

表 2: 小组成员分工

姓名	分工	主要贡献
李嘉明	基础框架搭建与核心逻辑	项目初始化、GameObject/Player/Enemy/Bullet 基础类、宝箱系统、道具系统、菜单 UI、武器更新、代码整理与合并
张怡馨	高级功能与系统完善	游戏状态机与菜单管理、最高分存档/持久化设置、武器升级系统、Boss 弹幕攻击、关卡递进系统、音效集成、鼠标操作支持、玩法打磨与 README
陈莹蔚	基础功能开发与游戏功能集成测试	敌人与碰撞计分、粒子特效/浮动文字; 整合 A/B/C 模块功能，完善整体游戏流程

4 项目总结与反思

4.1 项目成果

本项目成功实现了一款功能完整的竖版射击小游戏，涵盖了从基础战斗到高级 Boss 战、从武器升级到音效特效的全套游戏体验。项目采用 CMake + Qt 6 构建体系，代码组织清晰，模块划分合理。

技术亮点包括：

- **多态武器系统：**通过策略模式实现三种武器的灵活切换与独立升级树。
- **加权随机升级池：**宝箱升级采用权重算法，兼顾平衡性与随机性的趣味。
- **帧率无关的游戏循环：**所有运动基于 `deltaTime`，保证不同性能设备上的一致体验。
- **全过程化渲染：**所有图形均由 `QPainter` 绘制，无外部图片依赖，展示了对 Qt 绘图体系的深入理解。
- **完整的状态机管理：**Menu/Settings/Playing/Paused/GameOver 五状态清晰切换，交互流畅。

4.2 待改进之处

1. **代码架构：**当前大部分类以头文件内联实现（header-only），虽简化了编译但不符合常规 Qt 项目组织方式，大型项目维护时会增加编译时间。
2. **碰撞检测效率：**当前使用双重循环遍历所有实体对进行碰撞检测，实体数量多时可考虑空间分区（如网格/四叉树）优化。
3. **Boss 行为扩展：**Boss 弹幕样式可进一步丰富（如圆形弹幕、激光扫射等），增加视觉冲击力。
4. **性能监控：**缺少帧率显示和性能诊断手段，不便于定位性能瓶颈。
5. **跨平台测试：**项目在 Windows 下开发，未在其他平台（Linux/macOS）充分验证。

4.3 团队协作反思

1. **版本控制规范：**团队使用 Git + GitHub Pull Request 工作流进行协作，有效避免了代码冲突。但部分提交信息较为简略（如“上传 ui 文件”），建议统一提交信息格式。
2. **任务划分：**基础框架与高级功能的分工合理，但 PR 合并过程中出现了少量冲突，说明接口设计初期可更充分地沟通数据结构约定。
3. **文档同步：**设计文档与实际实现存在一定偏差（如“火力全开”“掷地有声”等技能未实现），建议在开发过程中及时更新设计文档，避免设计与实现脱节。
4. **测试覆盖：**项目缺乏自动化测试，主要依赖手动游玩验证，后续可引入单元测试框架提升代码质量保障。

4.4 学习收获

通过本项目，小组成员实践了以下技能：

- 基于 Qt 框架的 C++ GUI 应用程序开发
- 游戏引擎核心循环的设计与实现
- 面向对象设计模式（继承、多态、策略模式）的实际运用
- CMake 构建系统的配置与使用
- Git 团队协作工作流
- 游戏数值平衡与难度曲线设计

项目仓库地址：<https://github.com/noobFS114514/TheLevelTwo>